Winning Space Race with Data Science

Wing Li

July 1, 2026

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project

Outline

- Executive Summary
- Introduction
- Methodology
- Results
- Conclusion
- Appendix

Executive Summary

- Summary of methodologies

- Data collection
- Data wrangling
- EDA with data visualization
- EDA with SQL
- Building an interactive map with Folium
- Building a Dashboard with Plotly Dash
- Predictive analysis (Classification)

- Summary of all results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results

Introduction

- Project background and context

- The Cost Gap: Competitors charge $165M+ per launch-->SpaceX advertises the Falcon 9 at $62M.
- The Key Driver: Massive savings are achieved by reusing the first-stage booster core instead of expending it.
- The Business Value: Predicting landing success reveals the true cost of a launch, giving competitors a massive data advantage when bidding for rocket launch contracts.

- Key Problems We Are Solving

- ❓ What drives landing success? Analyzing how variables like payload mass, orbit type, and launch site location dictate recovery outcomes.
- 📈 How do rocket variables interact? Quantifying the exact impact of hardware configurations (e.g., grid fins, landing legs, core blocks) on success rates.
- 🎯 What are the optimal operational conditions? Determining the ideal thresholds SpaceX must maintain to guarantee predictable, high-probability booster recovery.

Section 1

Methodology

Methodology

Executive Summary

- Data collection methodology:
  - SpaceX REST API: Retrieved foundational launch records targeting [api.spacexdata.com/v4/] to gather data on rockets, payload masses, launch specifications, and landing outcomes.
  - Wikipedia Web Scraping: Extracted supplemental historical Falcon 9 and Falcon Heavy launch logs using BeautifulSoup from Wikipedia to ensure a complete, robust dataset.
- Perform data wrangling
  - Data Cleaning & Transformation: Combined the API and scraped datasets, handled missing values, and created a unified structure suitable for machine learning.
  - Feature Engineering: Applied One-Hot Encoding to categorical data fields and dropped non-predictive, irrelevant columns to optimize the final model training matrix.
- Perform exploratory data analysis (EDA) using visualization and SQL
  - Plotting : Scatter Graphs, Bar Graphs to show relationships between variables to show patterns of data
- Perform interactive visual analytics using Folium and Plotly Dash
- Perform predictive analysis using classification models
  - How to build, tune, evaluate classification models

Methodology

Data Sources & Extraction Workflows

- Target Data Attributes
  - SpaceX REST API Data Elements: Extracted comprehensive records regarding launch specifications, rockets utilized, payload masses delivered, landing site configurations, and core landing outcomes.
  - Core Predictive Goal: Built the target parameters to predict whether SpaceX will successfully attempt to land a rocket core based on these combined historical features.
- Data Collection Pipeline Workflows
  - SpaceX REST API Workflow:
    - Use SpaceX REST API → API returns SpaceX data in .JSON format → Normalize data into flat data file such as .csv
  - Wikipedia Web Scraping Workflow:
    - Get HTML Response from Wikipedia → Extract data using BeautifulSoup → Normalize data into flat data file such as .csv
- Perform Interactive Visual Analytics
  - Geospatial & Dynamic Mapping: Deployed Folium map visualizations and built an interactive web application dashboard using Plotly Dash to isolate and analyze launch patterns.
- Perform Predictive Analysis Using Classification Models
  - Machine Learning Pipeline: Designed, tuned, and evaluated predictive classification models to find the optimal algorithm for landing success prediction.

Data Collection

- How the data sets were collected:
  - Multi-Source Data Acquisition Strategy
    - Primary API Ingestion: Queried the official SpaceX REST API to retrieve comprehensive, structured launch logs, hardware configurations, and core landing histories.
    - Secondary Web Scraping: Parsed historical Wikipedia launch logs to isolate structural records and cross-verify missing parameters.
  - Predictive Analytical Objective
    - Targeted key features to predict first-stage booster landing success, enabling data-driven launch cost estimation (reusability reduces baseline launch costs from upward of $165M down to $62M).
  - Downstream Pipeline Integration
    - Combined both extraction channels into a unified preprocessing flow consisting of structural data cleaning, missing value imputation, and feature engineering.

Data Collection – SpaceX API

1) Endpoint Target: Requested historical telemetry records from the launches/past endpoint via [api.spacexdata.com/v4/]
2) Relational Feature Resolution: Used structural identification IDs within the initial payload to call auxiliary API routes (/rockets/, /launchpads/, /payloads/, and /cores/), mapping details like booster versions, coordinates, payload mass, and landing hardware
3) Data Isolation: Filtered the normalized dataset exclusively for Falcon 9 configurations (removing Falcon 1 records) and restricted data history up to November 13, 2020

[1. Request API JSON] ➔ [2. Normalize via pd.json_normalize()] ➔ [3. Query ID Routes via Custom Helper Functions] ➔ [4. Map Lists to Dictionary/DataFrame] ➔ [5. Filter for 'Falcon 9' & Reset Index]

GitHub URL: https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/jupyter-labs-spacex-data-collection-api.ipynb

Data Collection - Scraping

- Data Extraction & Parsing
  1. Fetch HTML Response: Request the raw HTML content from the target Wikipedia URL using requests.get().
  2. Initialize BeautifulSoup: Parse the response payload text into a navigable document object using the 'html.parser' engine.
  3. Locate Target Tables: Scan the DOM using soup.find_all('table') to target and isolate the relevant launch logs.
  4. Extract Table Headers: Iterate through <th> tags and apply custom string cleaning to gather verified, non-empty column names.

- Data Structuring & Flat Serialization
  5. Initialize Data Structure: Create an empty Python dictionary (launch_dict) keyed by the extracted headers, removing irrelevant variables and pre-populating target arrays
  6. Append Scraped Content: Loop through rows (<tr>) inside the target wikitable structures to parse, filter, and append raw text values to their corresponding dictionary lists.
  7. DataFrame Conversion: Structuralize the compiled data matrix into a flat Pandas format by converting the dictionary elements into individual pd.Series() vectors via pd.DataFrame
  8. Export to CSV: Serialize the finalized DataFrame to a static flat data file using df.to_csv('spacex_web_scraped.csv', index=False).

[HTML Request] ➔ [BeautifulSoup Object] ➔ [Isolate Tables] ➔ [Parse Headers & Rows] ➔ [Dictionary Mapping] ➔ [Pandas DataFrame] ➔ [.CSV Export]

GitHub URL: https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/jupyter-labs-webscraping.ipynb

Data Wrangling Methodology

- Introduction

Processed raw SpaceX telemetry logs to clean missing observations, standardize features, and transform qualitative records into strict mathematical structures suitable for predictive machine learning.

Core Data Engineering Operations:

• Launch Site Profiling: Isolated geographic trends across core landing hubs.

• Orbit Type Analysis: Mapped operational destination distribution frequencies.

• Outcome Label Transformation: Converted categorical parameters (True ASDS, False RTLS, None) into standard binary classification target vectors where 1 indicates successful booster recovery and 0 indicates landing failure.

• Success Rate Evaluation: Established foundational baseline dataset metrics.

• Data Export: Consolidated features to a standardized flat .csv data vector.

GitHub URL:

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/labs-jupyter-spacex-Data-wrangling.ipynb

EDA and Data Visualization Methodology

- Summary of what charts were plotted and why we used those charts

- Scatter Plots (Categorical Analytics)
  - These charts were plotted using Seaborn's catplot to map continuous features against categorical variables, revealing baseline flight behaviors and success distributions
  - Flight Number vs. Payload Mass
  - Flight Number vs. Launch Site
  - Payload Mass vs. Launch Site
  - Flight Number vs. Orbit Type
  - Payload Mass vs. Orbit Type

- **Bar Charts (Success Distributions)**
  - Success Rate vs. Orbit Type: Generated by grouping the dataset by Orbit profiles and calculating the mathematical mean of the binary Class target variable. This visualization explicitly identifies which orbital targets are historically safest for booster retrieval. It confirms that SSO, HEO, GEO, and ES-L1 are highly reliable profiles boasting near-perfect success records.
- Line Graphs (Temporal Trends)
  - Success Rate vs. Year: Created after building a custom helper function to isolate and parse the annual calendar data from the raw Date strings. By tracking the success rates chronologically using sns.lineplot, it explicitly maps out SpaceX's operational curve: the landing success rate steadily and continuously optimized upward from 2013 through 2020.

Github URL to notebook

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/edadataviz.ipynb

EDA with SQL

- The SQL queries we performed:
- Displaying the unique names of launch sites
- Filtering the first 5 records starting with 'CCA'
- Aggregating the total payload mass for 'NASA (CRS)'
- Calculating the average payload mass for booster 'F9 v1.1'

Basic aggregations and string pattern filtering allow us to quickly establish baseline operational metrics from the dataset.

- Advanced Fleet & Landing Metrics:
- Isolating the earliest date of a successful ground pad landing
- Identifying boosters with drone ship success carrying 4,000 to 6,000 kg
- Counting total successful vs. failed mission outcomes
- Using subqueries to find all boosters carrying the maximum payload mass

Subqueries and conditional counts help dissect landing safety variables, pinpoint fleet milestones, and isolate core booster limits.

- Temporal Trends & Rankings:
- Parsing dates to isolate month names, booster versions, and drone ship failures for 2015
- Ranking the frequencies of successful landing outcomes between 2010-06-04 and 2017-03-20 in descending order

Advanced date parsing and grouping criteria allow us to track historical recovery methods and rank which options proved most reliable over time.

GitHub URL: https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/jupyter-labs-eda-sql-coursera_sqllite.ipynb

Interactive Visual Analytics

- This section demonstrates the implementation of interactive geospatial mapping and real-time dashboard frameworks to analyze SpaceX launch site data profiles.

1. Geospatial & Proximity Mapping (Folium):

- Built comprehensive spatial maps utilizing folium.Circle boundaries, MarkerClusters, and Haversine distance tracking to evaluate proximity trends relative to launch site safety parameters.
- GitHub URL: https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/Interactive_Visual_Analytics_with_%20Folium_launch_site_location.ipynb

2. Operational Visual Dashboards (Plotly Dash):

- Constructed a fully interactive analytics application utilizing Flask, dcc.Dropdown, and dcc.RangeSlider elements to dynamically switch between cumulative and localized success ratios and continuous payload metrics.
- Github URL:

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/spacex-dash-app.py

Build an Interactive Map with Folium

Geographical Mapping: Pinned launch site coordinates using folium.Circle boundaries and DivIcon text labels to evaluate their physical surroundings.

Launch Outcome Clustering: Grouped dense launch records into a MarkerCluster(), automatically mapping successes to **green** and failures to **red** markers for rapid site safety audits.

Proximity & Distance Telemetry: Calculated real-world distances (KM) using the Haversine formula and drew folium.PolyLine vectors to surrounding landmarks to evaluate spatial placement patterns.

Key Regional Trends Discovered:

- Are launch sites in close proximity to railways? **No**
- Are launch sites in close proximity to highways? **No**
- Are launch sites in close proximity to coastlines? **Yes** *(for safe overwater trajectories)*
- Do launch sites keep a safe distance away from major cities? **Yes**
- GitHub URL:

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/Interactive_Visual_Analytics_with_%20Folium_launch_site_location.ipynb

Build a Dashboard with Plotly Dash

- **Geographical Launch Proportions:** Generated dynamic pie charts (`px.pie`) that switch display logic based on user input. Selecting "All Sites" charts the proportional distribution of successful launches across all operational hubs, while selecting an individual site isolates its distinct success-to-failure ratio.
- **Payload Outcome Scatter Graph:** Visualized the non-linear relationship between launch outcomes and payload mass using `px.scatter`. The plot maps two continuous variables simultaneously, displaying straightforward data points color-coded by specific booster versions and sized proportionally to the total cargo weight.
- **Interactive Filtering Elements:** Implemented a searchable dropdown dashboard menu (`dcc.Dropdown`) paired with an interactive payload range slider (`dcc.RangeSlider`) to cleanly bound data limits between minimum and maximum cargo thresholds.
- **Live Deployment Hosting:** Structured the underlying web framework using the Flask engine wrapped in Plotly Dash, deploying the application live 24/7 on PythonAnywhere to allow public user interaction and remote data audits.
- GitHub URL:

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/spacex-dash-app.py

Predictive Analysis (Classification)

- Building the Models: Loaded the SpaceX dataset into NumPy arrays and Pandas dataframes, standardized features via feature engineering, and split data into stratified training and testing sets containing 18 test samples.
- Evaluating & Improving Model: Trained four distinct classification algorithms—Logistic Regression, Support Vector Machine (SVM), Decision Tree, and k-Nearest Neighbors (KNN). Tuned hyperparameters using cross-validated GridSearchCV parameter sweeps and generated confusion matrices to visualize classification accuracy.
- Finding the Best Performer: Evaluated and compared all models using an integrated performance score dictionary. The optimized classifiers successfully converged on a tied peak test accuracy score, demonstrating robust and identical predictive capacity on the 18-sample validation baseline.
- **Model Development Process Flowchart**

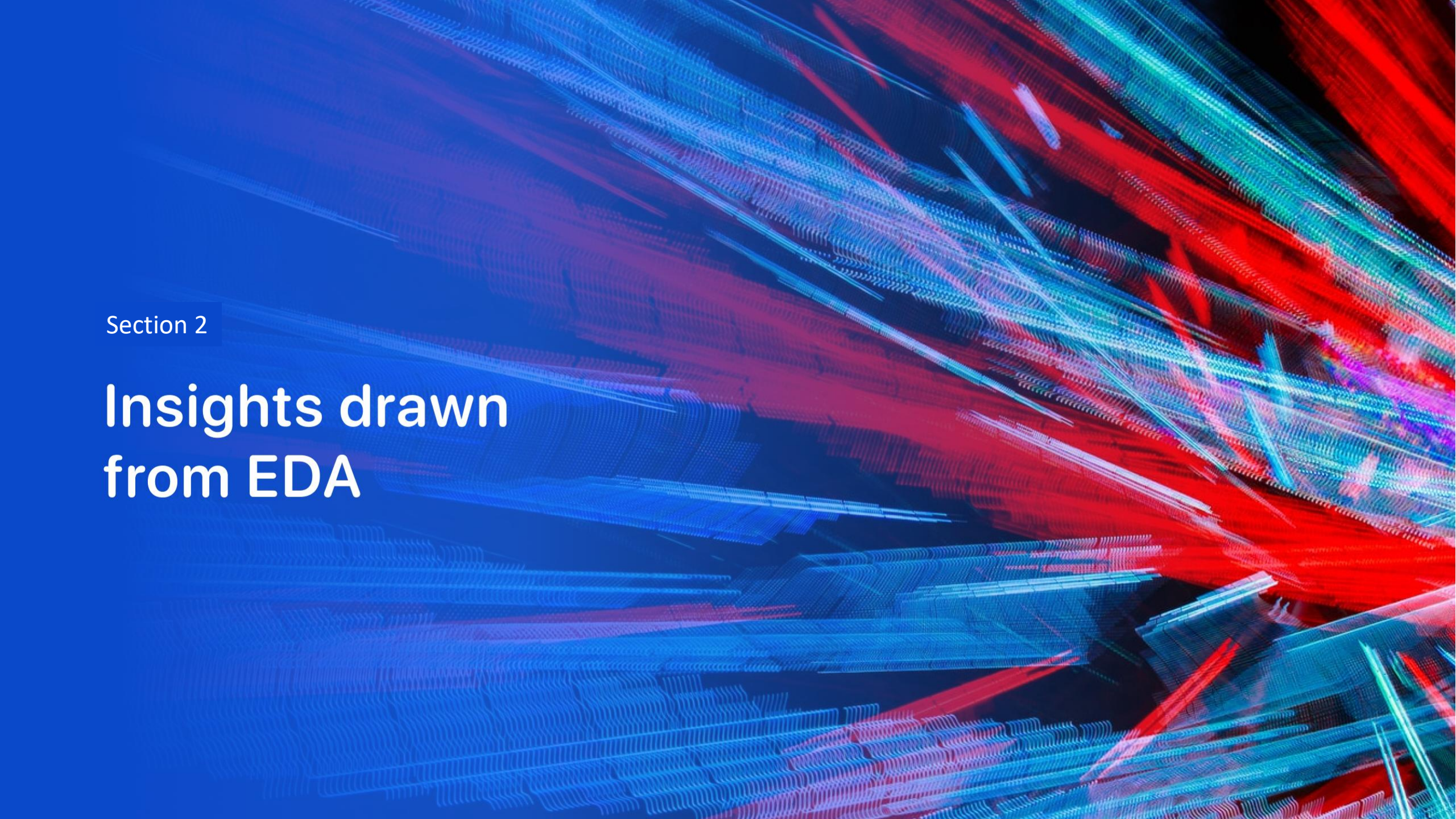
- Data Ingestion (NumPy/Pandas) → Standardization (StandardScaler) →Train/Test Split (18 Samples) → GridSearchCV Optimization → Algorithmic Evaluation

- GitHub URL:

https://github.com/Infinite-Sage/Applied-Data-Science-Capstone-Project/blob/main/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb

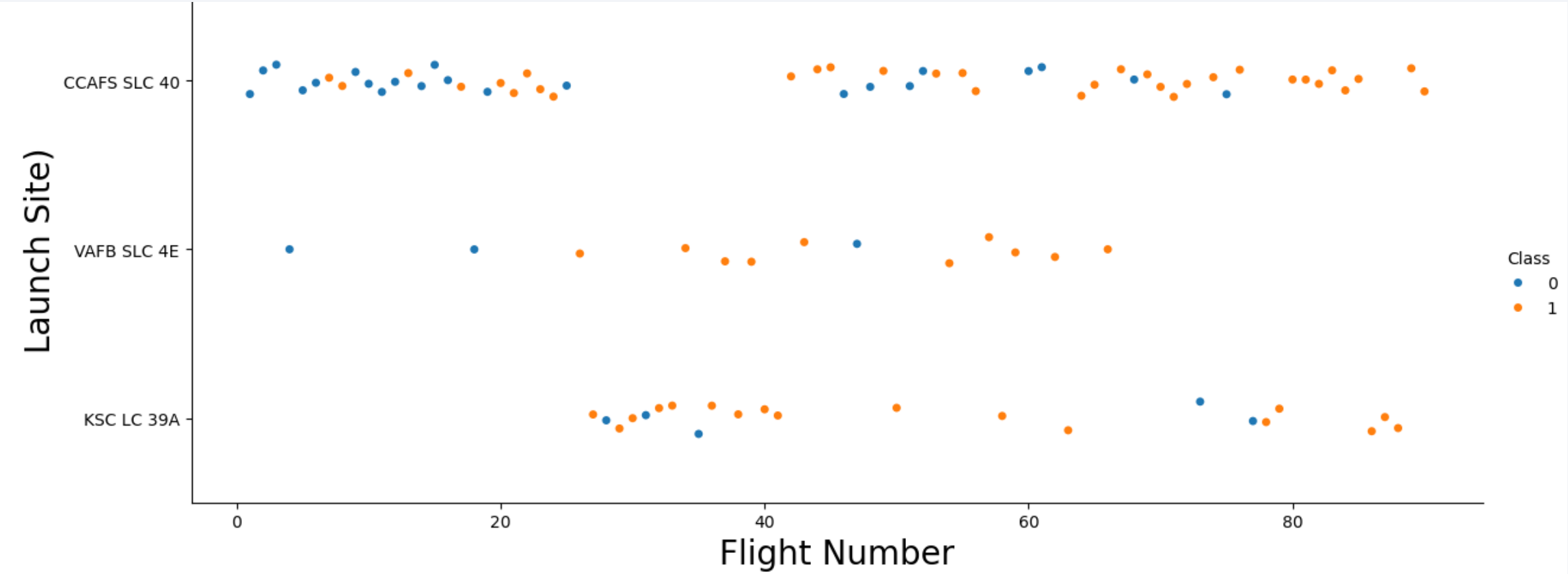
Results

- Exploratory data analysis results
- Interactive analytics demo in screenshots
- Predictive analysis results


Section 2
Insights drawn from EDA

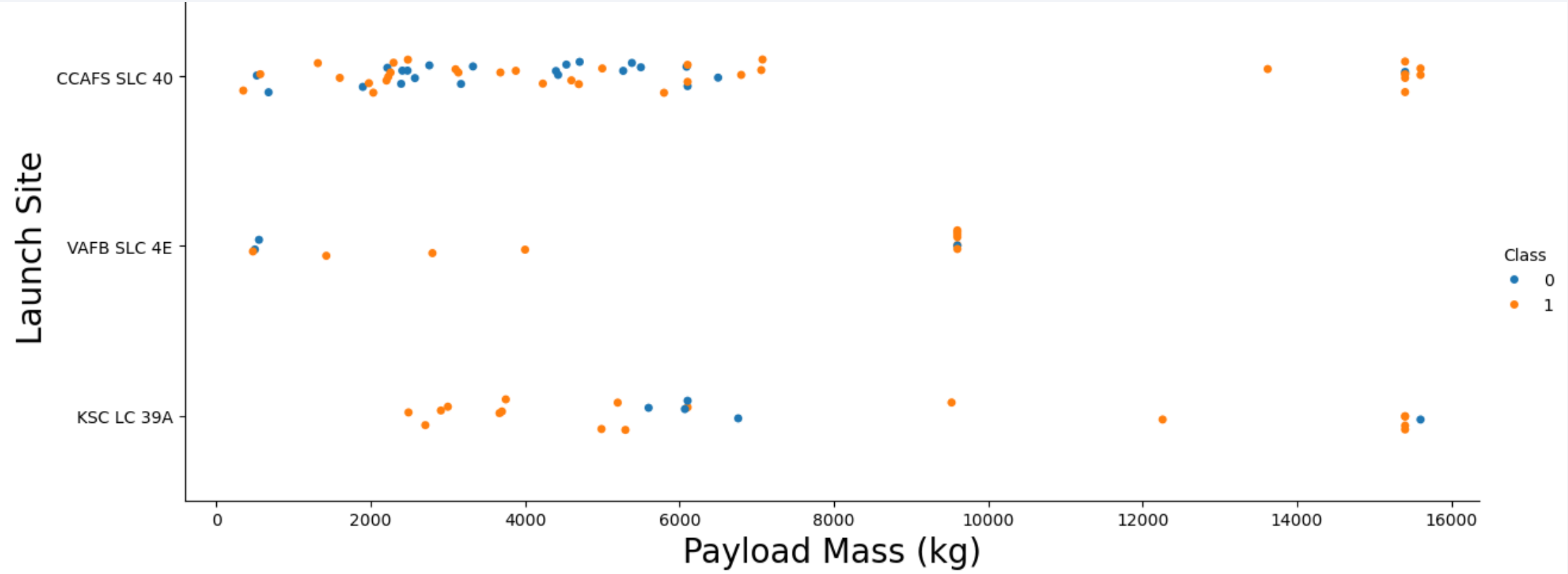
Flight Number vs. Launch Site

- We see that different launch sites have different success rates. CCAFS SLC-40, has a success rate of 60 %, while KSC LC-39A and VAFB SLC 4E has a success rate of 77%



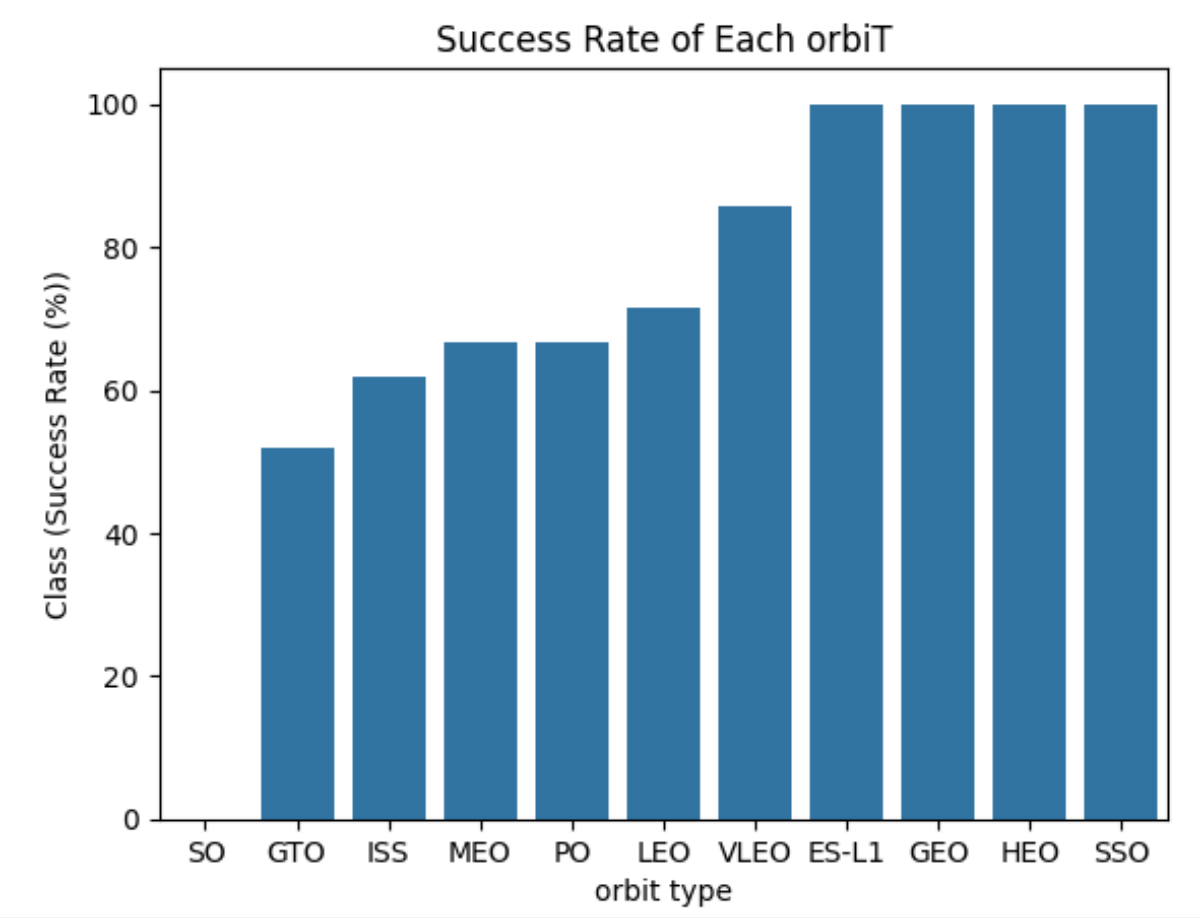
Payload vs. Launch Site

- For the VAFB-SLC launchsite there are no rockets launched for heavypayload mass(greater than 10000).
- The greater the payload mass for Launch Site CCAFS SLC 40 the higher the success rate for the Rocket.



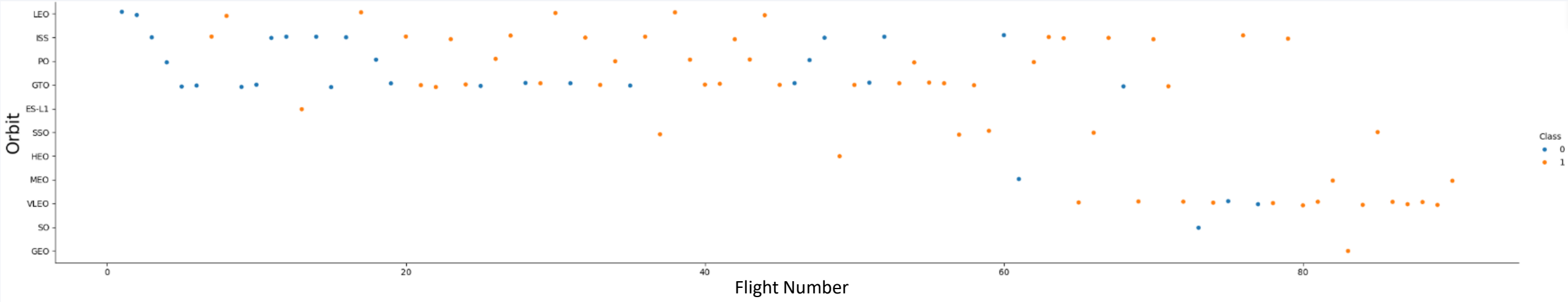
Success Rate vs. Orbit Type

- The Orbits SSO,HEO,GEO and ES-L1 are the successful orbits having High success rate.



Flight Number vs. Orbit Type

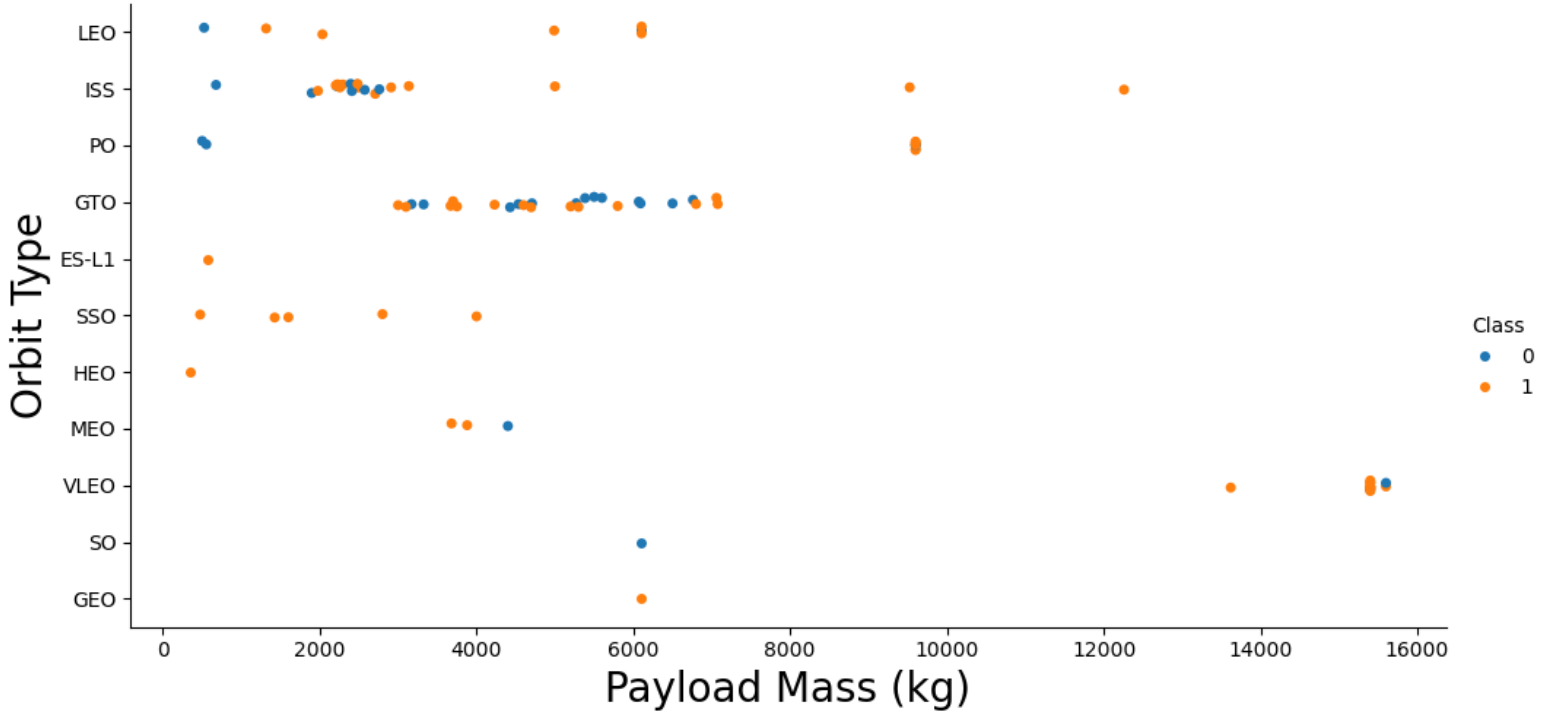
- In the LEO orbit, success seems to be related to the number of flights. Conversely, in the GTO orbit, there appears to be no relationship between flight number and success.



Payload vs. Orbit Type

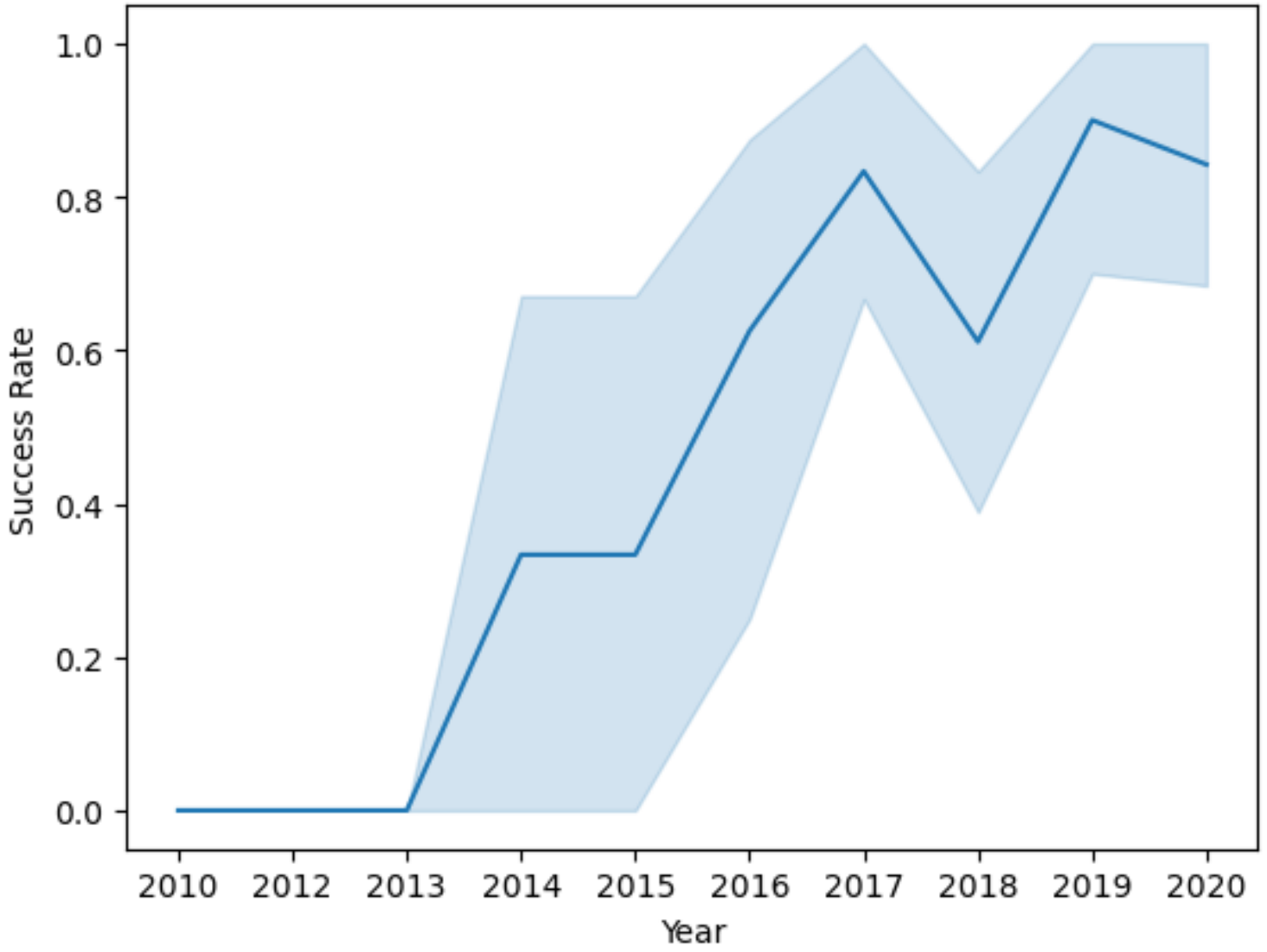
- With heavy payloads the successful landing or positive landing rate are more for Polar, LEO and ISS.

- However, for GTO, it's difficult to distinguish between successful and unsuccessful landings as both outcomes are present.



Launch Success Yearly Trend

We can observe that the sucess rate since 2013 kept increasing till 2020



EDA with SQL

Unique Launch Sites Names

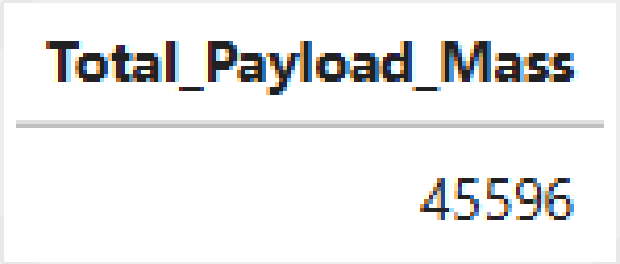
- SQL Query: SELECT DISTINCT launch_site FROM SPACEXTBL;
- QUERY EXPLAINATION: By applying the DISTINCT keyword to the launch_site column, the query filters out repetitive mission logs to provide a clean baseline inventory of SpaceX's primary launch facilities.

Launch_Site
CCAFS LC-40
VAFB SLC-4E
KSC LC-39A
CCAFS SLC-40

Launch Site Names Begin with 'CCA'

- SQL Query: SELECT * FROM SPACEXTABLE WHERE "Launch_Site" LIKE 'CCA%' LIMIT 5;
- QUERY EXPLAINATION
- Using the LIKE keyword with the 'CCA%' wildcard filters the dataset to return only missions launched from Cape Canaveral sites. The LIMIT 5 clause restricts the output to display the first 5 chronological launch records matching this criteria.

Date	Time (UTC)	Booster_Version	Launch_Site	Payload	PAYLOAD_MASS__KG_	Orbit	Customer	Mission_Outcome	Landing_Outcome
2010-06-04	18:45:00	F9 v1.0 B0003	CCAFS LC-40	Dragon Spacecraft Qualification Unit	0	LEO	SpaceX	Success	Failure (parachute)
2010-12-08	15:43:00	F9 v1.0 B0004	CCAFS LC-40	Dragon demo flight C1, two CubeSats, barrel of Brouere cheese	0	LEO (ISS)	NASA (COTS) NRO	Success	Failure (parachute)
2012-05-22	7:44:00	F9 v1.0 B0005	CCAFS LC-40	Dragon demo flight C2	525	LEO (ISS)	NASA (COTS)	Success	No attempt
2012-10-08	0:35:00	F9 v1.0 B0006	CCAFS LC-40	SpaceX CRS-1	500	LEO (ISS)	NASA (CRS)	Success	No attempt
2013-03-01	15:10:00	F9 v1.0 B0007	CCAFS LC-40	SpaceX CRS-2	677	LEO (ISS)	NASA (CRS)	Success	No attempt



Total Payload Mass Carried by boosters from NASA (CRS)

- SQL Query: SELECT SUM("Payload_Mass__kg_") AS Total_Payload_Mass FROM SPACEXTABLE WHERE "Customer" = 'NASA (CRS)';
- QUERY EXPLAINATION: The query utilizes the SUM() aggregate function to calculate the cumulative weight of all cargo transported in the dataset. By applying a WHERE clause, it restricts the calculation exclusively to missions commissioned by 'NASA (CRS)', returning a total combined payload mass of 45,596 kg

Average Payload Mass carried by booster version F9 v1.1

- SQL Query: SELECT AVG("Payload_Mass__kg_") AS Avg_Payload_Mass FROM SPACEXTABLE WHERE "Booster_Version" = 'F9 v1.1';
- QUERY EXPLAINATION: The query applies the **AVG()** aggregate function to calculate the mean mass of all cargo carried across the fleet. The **WHERE** clause isolates rows where the booster version is explicitly **'F9 v1.1'**, filtering out other models to output an average payload of **2,928.4 kg**

Avg_Payload_Mass
2928.4

The date when the first successful landing outcome in ground pad was achieved

- SQL Query: SELECT MIN("Date") AS First_Successful_Ground_Pad_Landing FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (ground pad)';
- QUERY EXPLAINATION: The query utilizes the MIN() aggregate function to locate the earliest chronological timestamp in the database. By applying the WHERE clause filter for 'Success (ground pad)', it isolates the precise calendar date when SpaceX achieved its very first successful land-based booster recovery (2015-12-22)

First_Successful_Ground_Pad_Landing
2015-12-22

Booster_Version
F9 FT B1022
F9 FT B1026
F9 FT B1021.2
F9 FT B1031.2

Successful Drone Ship Landing with Payload between 4000 and 6000

- SQL QUERY: SELECT DISTINCT "Booster_Version" FROM SPACEXTABLE WHERE "Landing_Outcome" = 'Success (drone ship)' AND "PAYLOAD_MASS__KG_" > 4000 AND "PAYLOAD_MASS__KG_" < 6000;
- The query uses the DISTINCT keyword to isolate unique booster versions without displaying duplicate configurations. By applying conditional AND operators, it evaluates multiple criteria simultaneously—filtering for a 'Success (drone ship)' landing outcome alongside a specific cargo threshold strictly between 4,000 kg and 6,000 kg.

Total Number of Successful and Failure Mission Outcomes

- QUERY EXPLANATION:The query utilizes independent **SELECT COUNT()** statements paired with string pattern matching to aggregate total fleet performance. By applying the **LIKE '%Success%'** and **LIKE '%Failure%'** wildcards to the mission outcome column, it simultaneously tallies and prints two separate metrics: total successful outcomes (**100**) versus total failed outcomes (**1**)

SQL Query:

```
SELECT COUNT(Mission_Outcome) AS SuccessOutcome FROM SPACEXTABLE WHERE
Mission_Outcome LIKE 'Success%';

SELECT COUNT(Mission_Outcome) AS FailureOutcome FROM SPACEXTABLE WHERE
Mission_Outcome LIKE 'Failure%';
```

SuccessOutcome
100

FailureOutcome
1

Boosters Carried Maximum Payload

- SQL QUERY:
- select PAYLOAD_MASS_KG_,Booster_version as boosterverssion from spacextbl where PAYLOAD_MASS_KG_ = (select max(PAYLOAD_MASS_KG_) from SPACEXTBL) ORDER BY Booster_version;
- Query Explanation:
- The query utilizes a nested scalar subquery containing the **MAX()** aggregate function to dynamically find the absolute highest weight value within the dataset. The primary **WHERE** clause then cross-references this ceiling value, filtering the entire table to extract and list every individual booster version that successfully carried that maximum peak capacity load (**15,600 kg**).

PAYLOAD_MASS__KG_	boosterverssion
15600	F9 B5 B1048.4
15600	F9 B5 B1048.5
15600	F9 B5 B1049.4
15600	F9 B5 B1049.5
15600	F9 B5 B1049.7
15600	F9 B5 B1051.3
15600	F9 B5 B1051.4
15600	F9 B5 B1051.6
15600	F9 B5 B1056.4
15600	F9 B5 B1058.3
15600	F9 B5 B1060.2
15600	F9 B5 B1060.3

2015 Launch Records

- SQL QUERY:

```
SELECT
    CASE SUBSTR(DATE, 6, 2)
        WHEN '01' THEN 'January'
        WHEN '02' THEN 'February'
        WHEN '03' THEN 'March'
        WHEN '04' THEN 'April'
        WHEN '05' THEN 'May'
        WHEN '06' THEN 'June'
        WHEN '07' THEN 'July'
        WHEN '08' THEN 'August'
        WHEN '09' THEN 'September'
        WHEN '10' THEN 'October'
        WHEN '11' THEN 'November'
        WHEN '12' THEN 'December'
    END AS Month_Name,
    LANDING_OUTCOME,
    BOOSTER_VERSION,
    LAUNCH_SITE
FROM SPACEXTBL
WHERE SUBSTR(DATE, 0, 5) = '2015'
  AND LANDING_OUTCOME = 'Failure (drone ship)';
```

- QUERY EXPLANATION:

The query utilizes a **CASE** statement combined with **SUBSTR()** string splicing to convert numeric text components into readable calendar month names. The primary **WHERE** clause applies multiple conditions simultaneously—filtering the dataset to extract specifically the year **'2015'** while isolating records where the recovery attempt resulted in a **'Failure (drone ship)'** outcome.

Month_Name	Landing_Outcome	Booster_Version	Launch_Site
January	Failure (drone ship)	F9 v1.1 B1012	CCAFS LC-40
April	Failure (drone ship)	F9 v1.1 B1015	CCAFS LC-40

Rank Landing Outcomes Between 2010-06-04 and 2017-03-20

- Query Explanation: The query utilizes the **COUNT()** function to calculate total frequencies grouped by distinct landing profiles. It employs a **BETWEEN** operator to constrain the search to a specific chronological date range, filters for successful outcomes using a **LIKE** wildcard pattern, and applies an **ORDER BY ... DESC** clause to rank the final counts in descending order.
- SQL QUERY:

```
SELECT Landing_Outcome, COUNT(Landing_Outcome) AS Total_Count

FROM SPACEXTABLE

WHERE DATE(Date) BETWEEN DATE('2010-06-04') AND DATE('2017-03-20')

    AND Landing_Outcome LIKE 'Success%'

GROUP BY Landing_Outcome

ORDER BY Total_Count DESC;
```

Landing_Outcome	Total_Count
Success (drone ship)	5
Success (ground pad)	3

Section 3

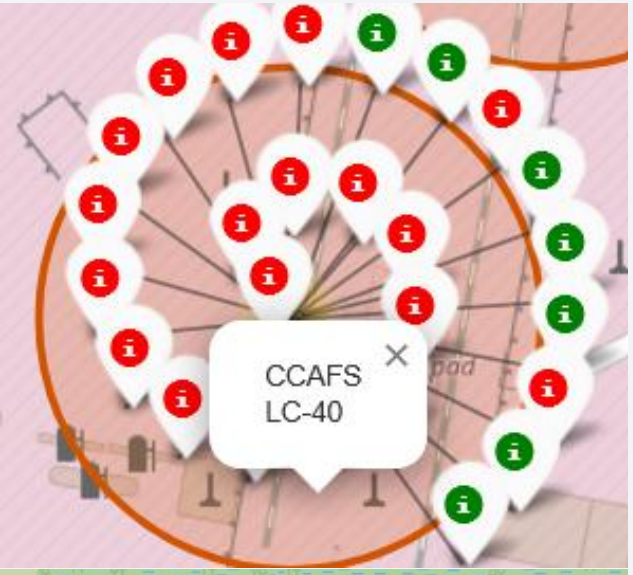
Launch Sites Proximities Analysis

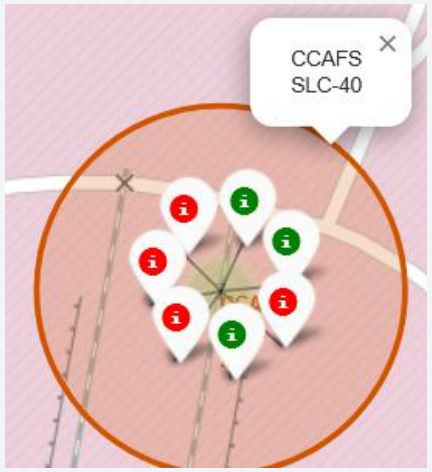
All Launch Sites Global Map Markers

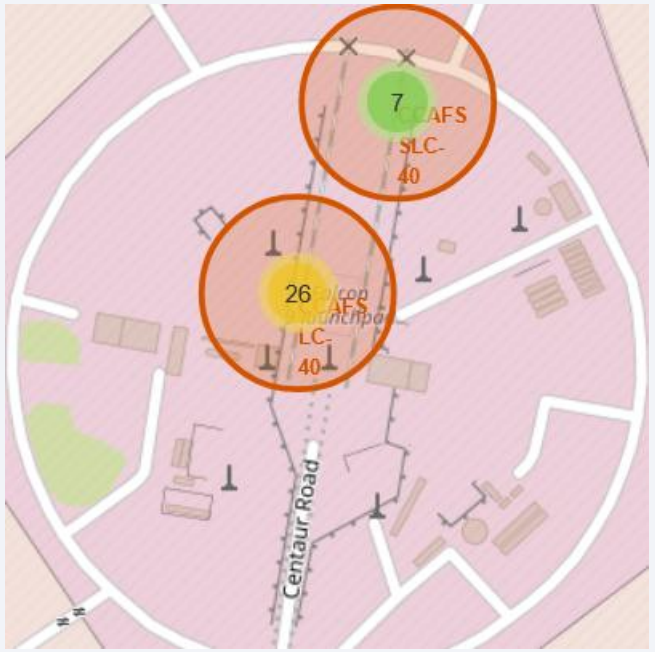
The SpaceX launch sites are in the United States of America coasts. Florida and California

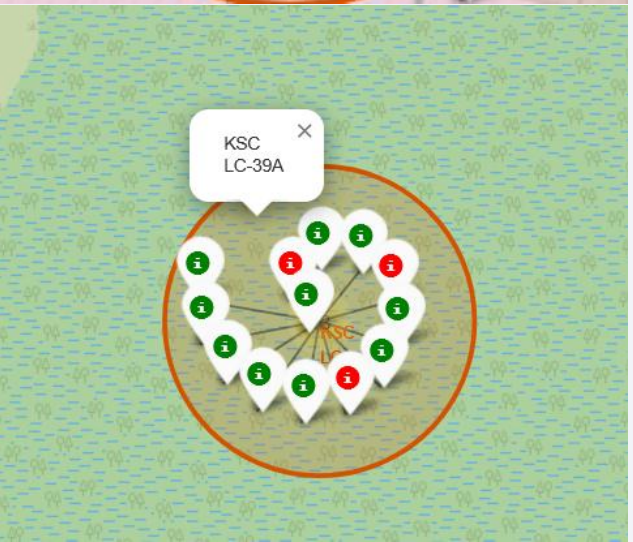
Color Labelled Markers

Florida Launch Sites





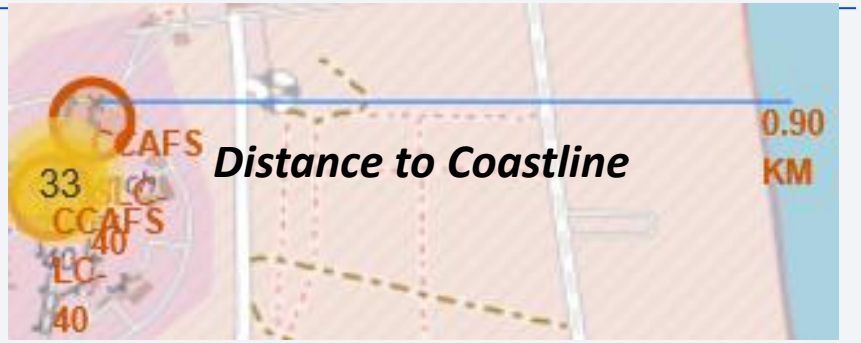




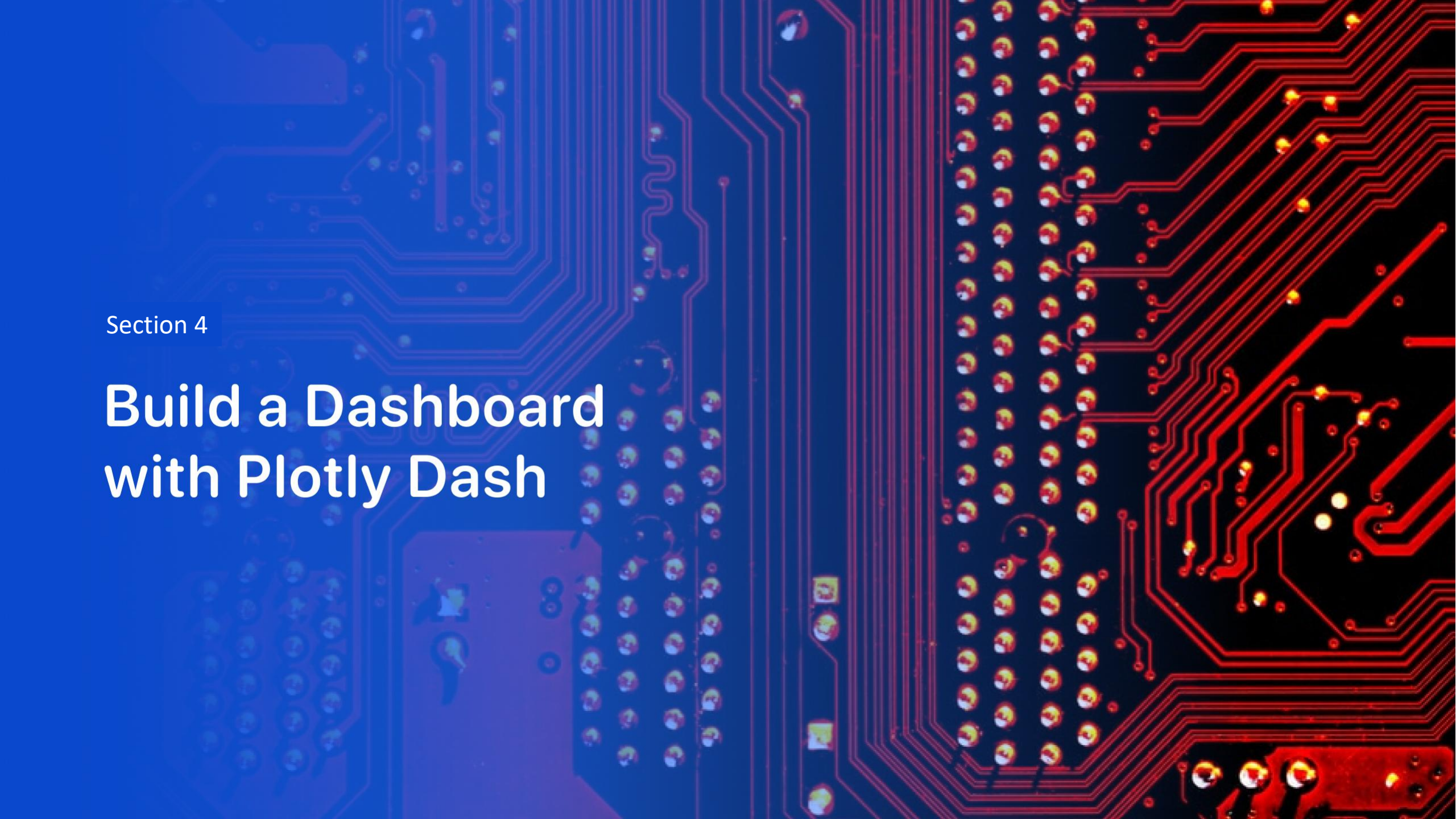
Green marker shows successful launch, and a Red marker shows a failed launch.

California Launch Site

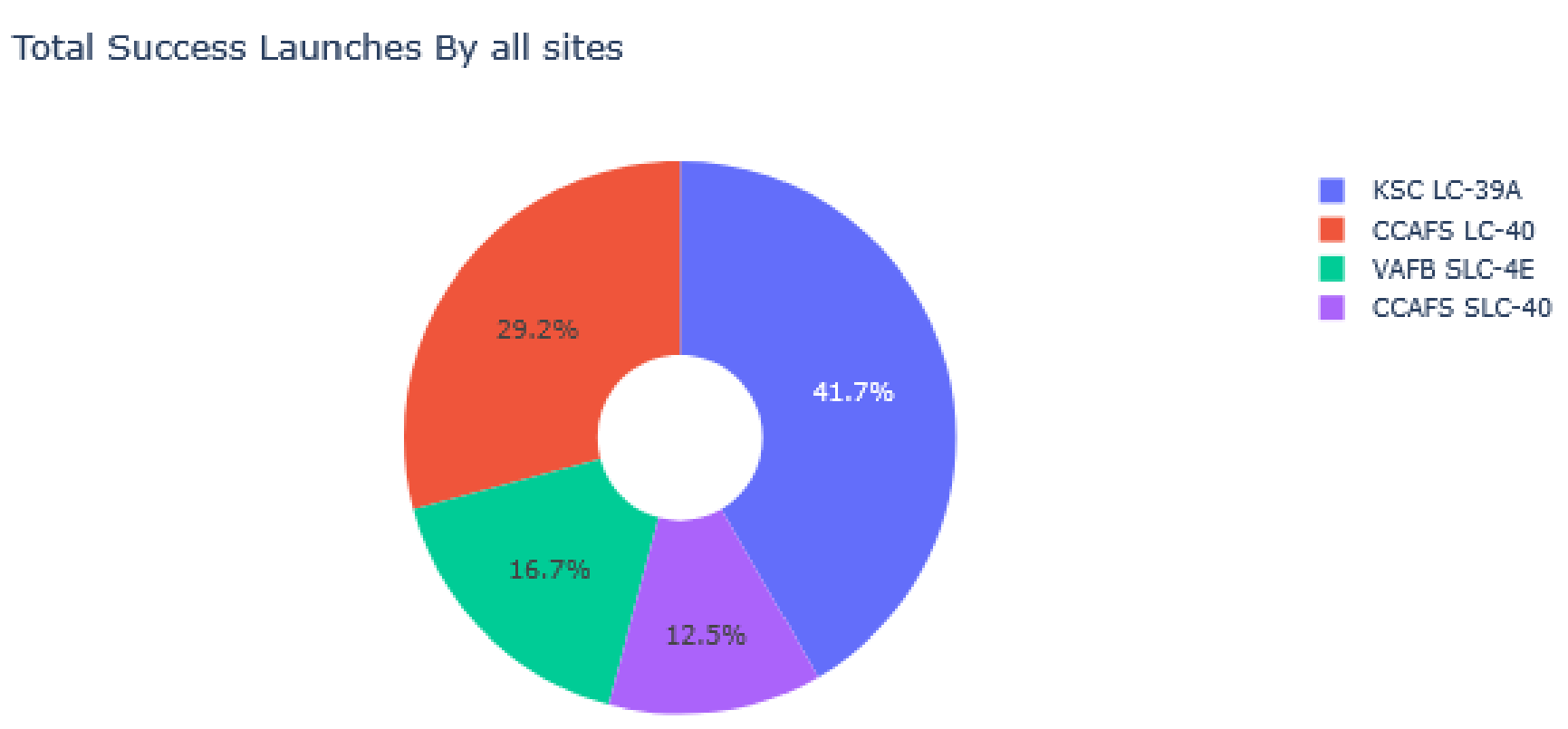
Landmark Distance Analysis with Haversine formula (CCAFS SLC-40 Reference)



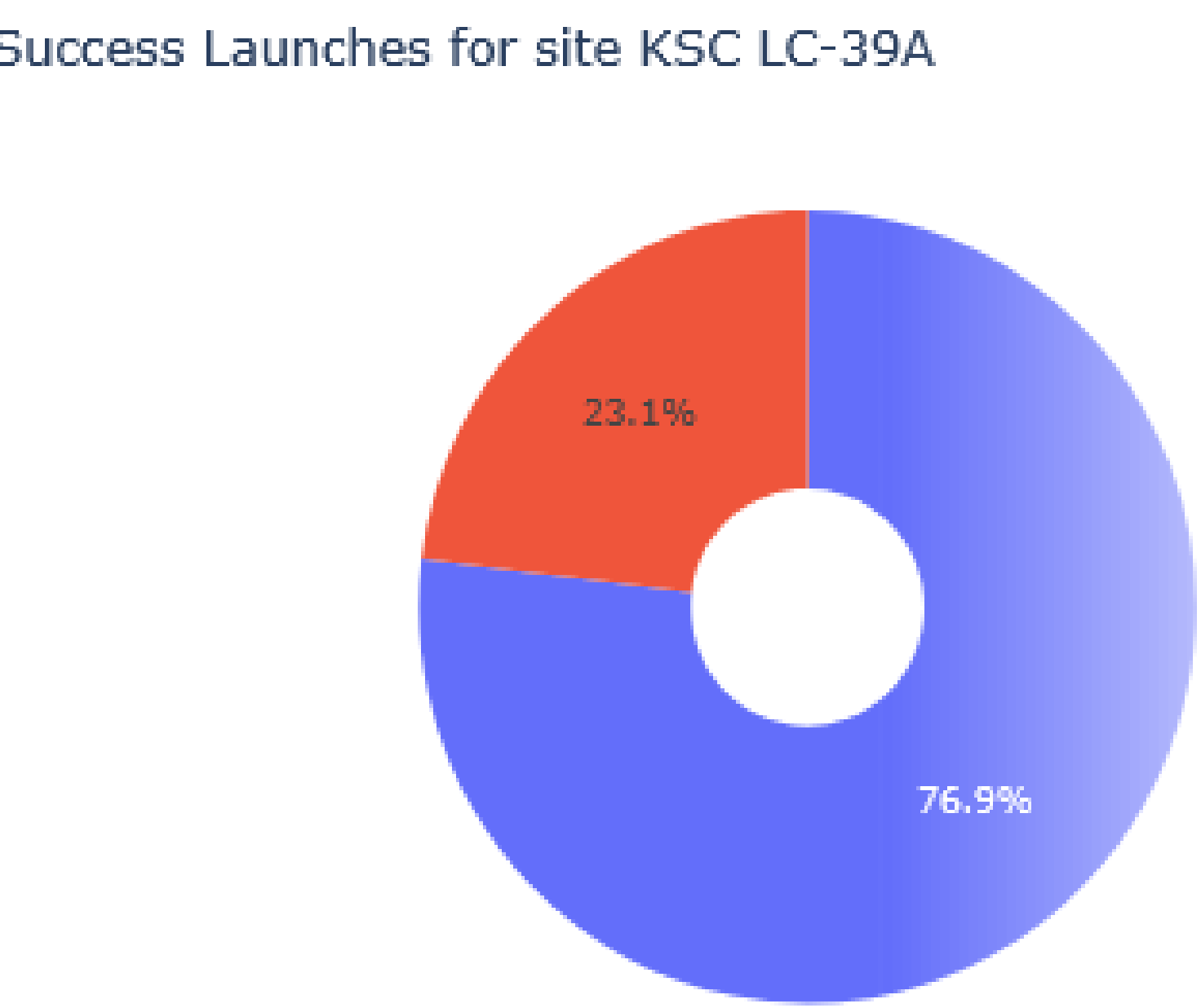
- Are launch sites in close proximity to railways? No
- Are launch sites in close proximity to highways? No
- Are launch sites in close proximity to coastline? Yes
- Do launch sites keep certain distance away from cities? Yes


Section 4
Build a Dashboard with Plotly Dash

SpaceX Interactive Launch Records Dashboard: All Sites View



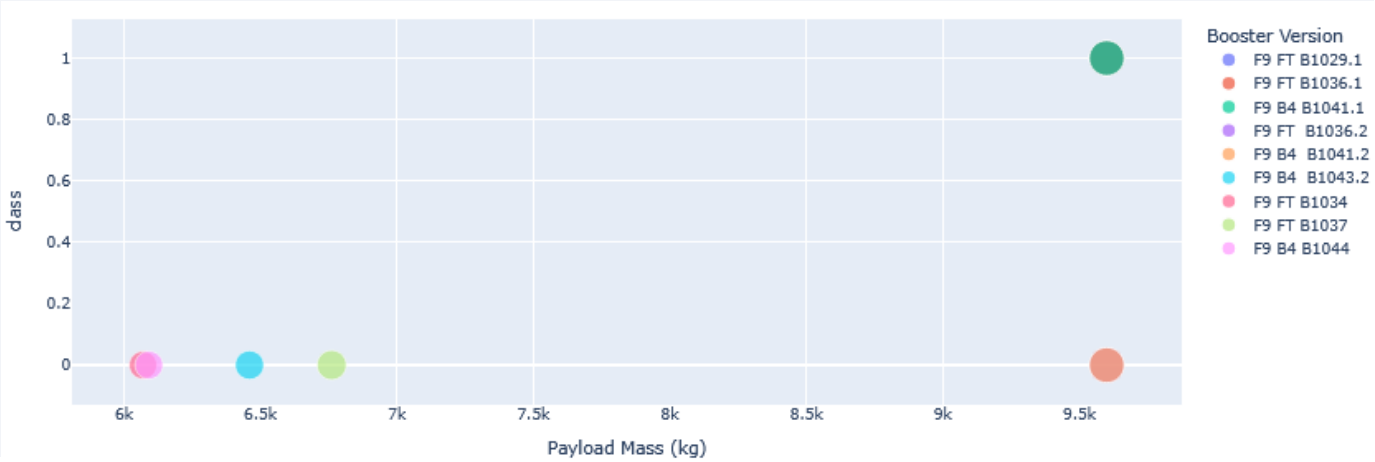
KSC LC-39A has the most successful launches among all the sites

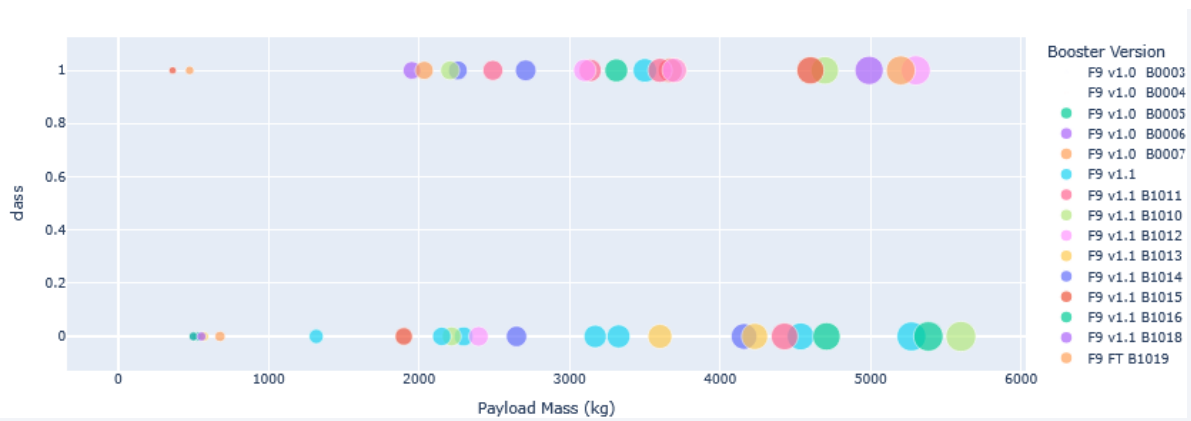


Dashboard Analysis: Launch Site with Highest Launch Success Ratio (KSC LC-39A)

KSC LC-39A achieved a 76.9% success rate while getting a 23.1% failure rate

Dashboard Analysis: Payload Mass vs. Launch Success Correlations



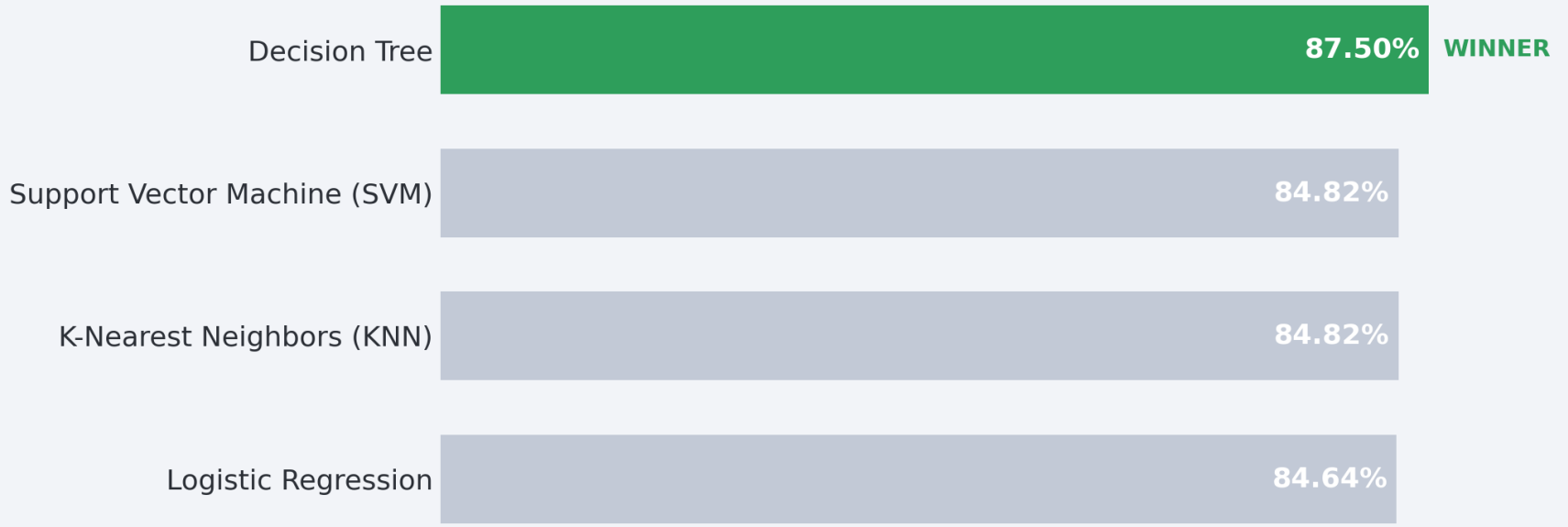


- The data indicates that mid-to-heavy payloads (2,000 kg to 5,500 kg) demonstrate a higher success rate compared to low-weight payloads (under 2,000 kg), which show a significant cluster of landing failures.

Section 5

Predictive Analysis (Classification)

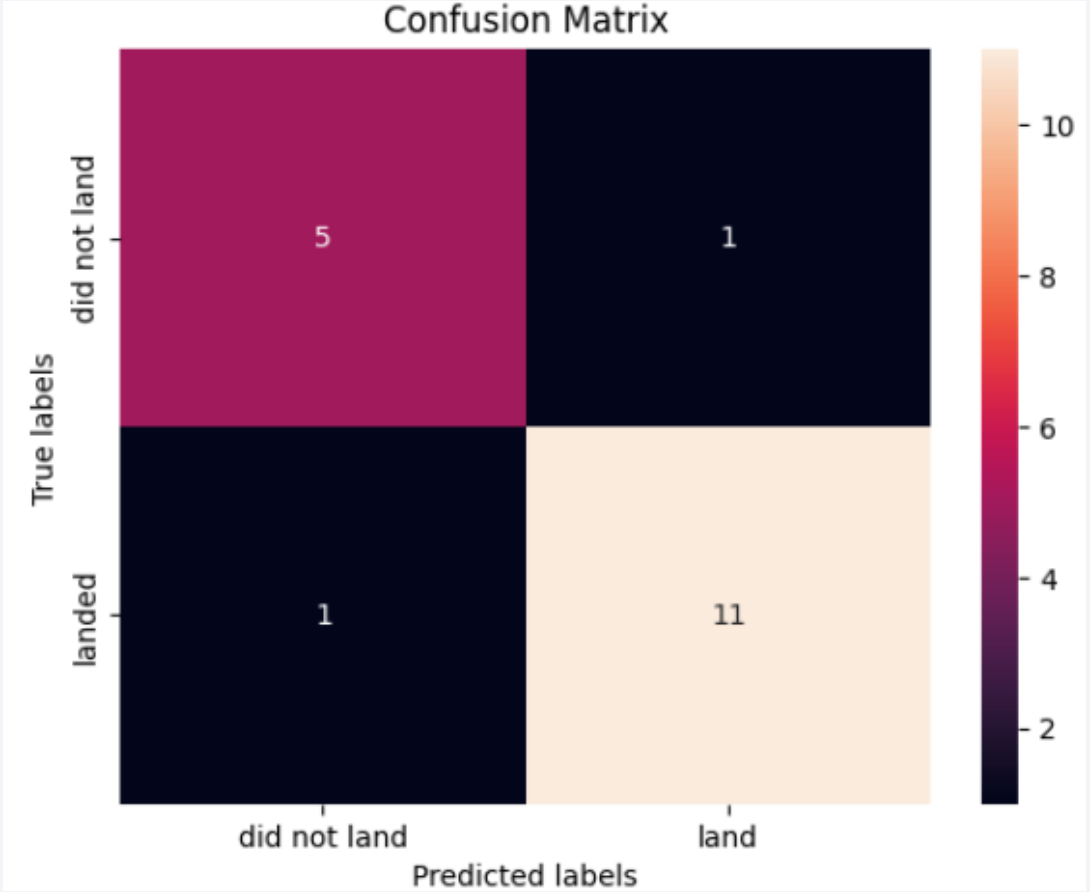
Classification Accuracy



Algorithm	Accuracy
Decision Tree (Winner)	**87.50%**
Support Vector Machine (SVM)	84.82%
K-Nearest Neighbors (KNN)	84.82%
Logistic Regression	84.64%

Confusion Matrix of the best performing model (Decision Tree)

- The major problem of the Decision Tree is the FP and FN.



Conclusions

- **Optimal Predictive Framework:** Based on the model evaluation, the **Decision Tree Classifier** is the best performing model for this dataset, achieving a peak training validation score of **87.50%** and an out-of-sample test accuracy of **88.89%**.
- **Payload Success Thresholds:** Exploratory data analysis reveals that **mid-to-heavy payloads (2,000 kg to 5,500 kg)** demonstrate a significantly higher landing success rate compared to low-weight payloads (under 2,000 kg), which saw a dense cluster of landing failures.
- **Launch Site Dominance:** Geographic exploratory data analysis identifies **KSC LC-39A** as the most successful operational launch site across the SpaceX fleet.
- **Orbit Profiles:** Target destination profiles heading to **GEO, HEO, SSO, and ES-L1** orbits maintain the highest historical landing success rates.
- **Economic Advantage:** Successfully predicting first-stage recoveries validates SpaceX's competitive **$62M** launch pricing model against traditional providers (costing upward of **$165M**), offering critical data-driven leverage for alternative companies bidding on rocket launches.

Appendix

- Haversine formula

Appendix

- **Appendix: Geographic Distance Calculations**
- **The Haversine Formula:** Used to calculate the great-circle distance between two points on a sphere given their longitudes and latitudes. This is critical for mapping launch pad coordinates to landing zones.
- **Mathematical Model:**
- $$a = \sin^2\left(\frac{\Delta\phi}{2}\right) + \cos(\phi_1) \cdot \cos(\phi_2) \cdot \sin^2\left(\frac{\Delta\lambda}{2}\right)$$
- $$c = 2 \cdot \operatorname{atan2}\left(\sqrt{a}, \sqrt{1-a}\right)$$
- $$d = R \cdot c$$
- **Variables Defined:**
  - d: The calculated distance between the two coordinates along the Earth's surface.
  - R: The radius of the Earth (mean radius = 6,371 km).
  - ϕ_1, ϕ_2: Latitude of point 1 and point 2 (converted into radians).
  - λ_1, λ_2: Longitude of point 1 and point 2 (converted into radians).
- **Application:** Implemented via a custom Python function to calculate spatial proximity from launch sites to coastal recovery zones, providing critical features for the model's landing predictive accuracy.

Thank you!